Лабораторная работа №1

Тема: Анализ предметной области

Цель: изучение бизнес-процесса и информационных потоков, имеющих место в заданной предметной области, освоение средств моделирования предметной области в виде инфологической модели данных и базовых моделей UML.

Задание

1. Изучить заданный массив входной информации (приложение А): структурировать его по ключевым сущностям с указанием всех атрибутов.
2. Детально рассмотреть структурированный в п.1 информационный массив и разработать инфологическую модель данных «сущность-связь».
3. Выделить и описать входные и выходные потоки информации (источники сведений, возможную форму представления, частоту поступления либо обновления, трудоемкость обработки). Составить блок-схему инфопотоков.
4. Выполнить описание процедур преобразования входных инфопотоков в выходные. Построить базовые диаграммы UML:
   - диаграмму вариантов использования – отражать только базовые варианты, число действующих лиц не менее двух;
   - диаграмму классов – соблюсти соответствие а) классов-сущностей сущностям модели данных, построенной в п.2, б) граничных классов действующим лицам, заполнять только секции имени и атрибутов.
5. Выполнить в табличной форме краткое описание вариантов использования и классов, разработанных в п.4.

В отчет включить: структурированный массив входной информации, описания инфопотоков и процедур их преобразования, построенные схемы и модели с описанием, выводы по работе.

Методические указания к выполнению

ER-диаграмма

Схема «сущность-связь» (также ERD или ER-диаграмма) — это разновидность блок-схемы, где показано, как разные «сущности» (люди, объекты, концепции и так далее) связаны между собой внутри системы. ER-диаграммы чаще всего применяются для проектирования и отладки баз данных для разработки программного обеспечения и информационных систем для бизнеса. ER-диаграммы (или ER-модели) полагаются на стандартный набор символов, включая прямоугольники, ромбы, овалы и соединительные линии, для отображения сущностей, их атрибутов и связей. Эти диаграммы устроены по тому же принципу, что и грамматические структуры: сущности выполняют роль существительных, а связи — глаголов.

Символы ERD-сущностей

Под понятием «сущности» подразумеваются объекты или понятия, несущие важную информацию. С точки зрения грамматики, они, как правило, обозначаются существительными, например, «товар», «клиент», «заведение» или «промоакция». В табл. 1.1 представлены три наиболее распространенных типа сущностей, используемых в ER-диаграммах.

Таблица 1.1 – Символы ERD-сущностей

Символ	Название	Описание
Entity	Независимая (сильная) сущность	Не зависит от других сущностей и часто называется «родительской», так как у нее в подчинении обычно находятся слабые сущности. Независимые сущности сопровождаются первичным ключом, который позволяет идентифицировать каждый экземпляр сущности.

Символ	Название	Описание
Weak Entity	Зависимая (слабая) сущность	Сущность, которая зависит от сущности другого типа. Не сопровождается первичным ключом и не имеет значения в схеме без своей родительской сущности.
Associative Entity	Ассоциативная сущность	Соединяет экземпляры сущностей разных типов. Также содержит атрибуты, характерные для связей между этими сущностями.

Символы ERD-связей

Связи используются в схемах «сущность-связь» для обозначения взаимодействия между двумя сущностями. Грамматически связи, как правило, выражаются глаголами, например, «назначить», «закрепить», «отследить», и несут полезную информацию, которую невозможно получить, опираясь только на типы сущностей (табл. 1.2).

Таблица 1.2 – Символы ERD-связей

Символ	Название	Описание
Relationship	Связь	Отношение между сущностями.
Weak Relationship	Слабая связь	Связь между зависимой сущностью и ее «хозяином».

Символы ERD-атрибутов

ERD-атрибуты характеризуют сущности, позволяя пользователям лучше разобраться в устройстве базы данных. Атрибуты содержат информацию о сущностях, выделенных в концептуальной ER-диаграмме (табл. 1.3).

Таблица 1.3 – Символы ERD-атрибутов

Символ	Название	Описание
Attribute	Атрибут	Характеризует сущность, а также отношения между двумя или более элементами.
Multivalued Attribute	Многозначный атрибут	Атрибут, которому может быть присвоено несколько значений.
Derived Attribute	Производный атрибут	Атрибут, чье значение можно вычислить, опираясь на значения связанных с ним атрибутов.

Кардинальность и ординальность отношений

Кардинальность – максимальное число связей, которое может быть установлено между экземплярами разных сущностей.

Ординальность – минимальное количество связей между экземплярами двух сущностей.

Кардинальность и ординальность отображаются на соединительных линиях согласно выбранному формату нотации.

Возможные вариации типов связей и отношений между сущностями показаны на рис. 1.1

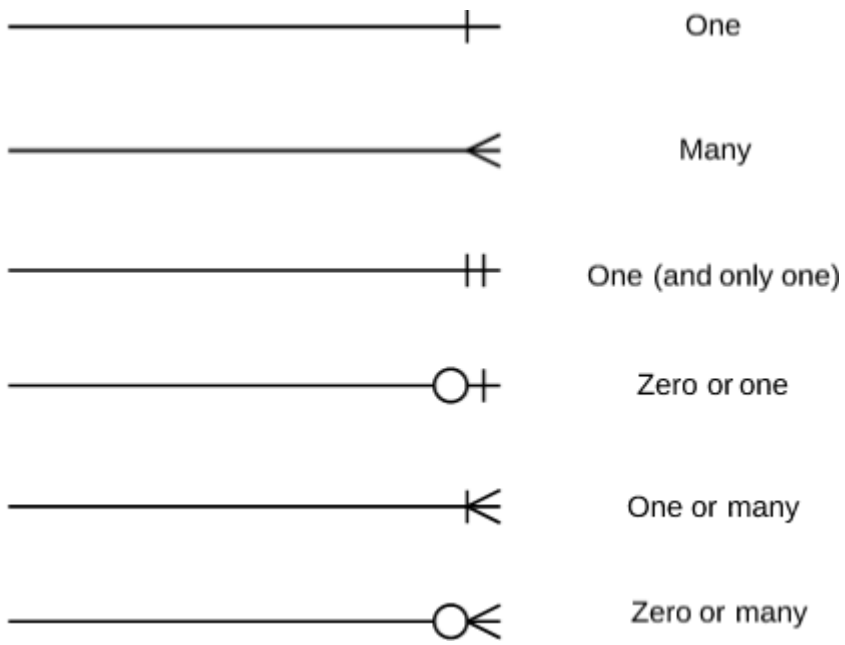


Рисунок 1.1 – Типы связей между сущностями и их графическое отображение

Технология разработки ER-диаграммы

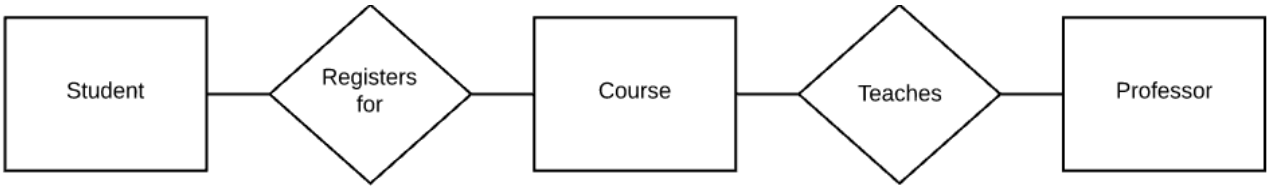
1. Задать сущности (они, как правило, обозначаются существительными, например, «автомобиль», «банк», «студент», «товар» и так далее)

Сущности – важнейшая составляющая ER-диаграммы. Рассмотрим пример построения концептуальной ER-диаграммы простой системы, где студент записывается на курс к профессору. Три основных сущности – это «студент», «курс» и «профессор»:

2. Определить связи (они показывают, как сущности взаимодействуют между собой)

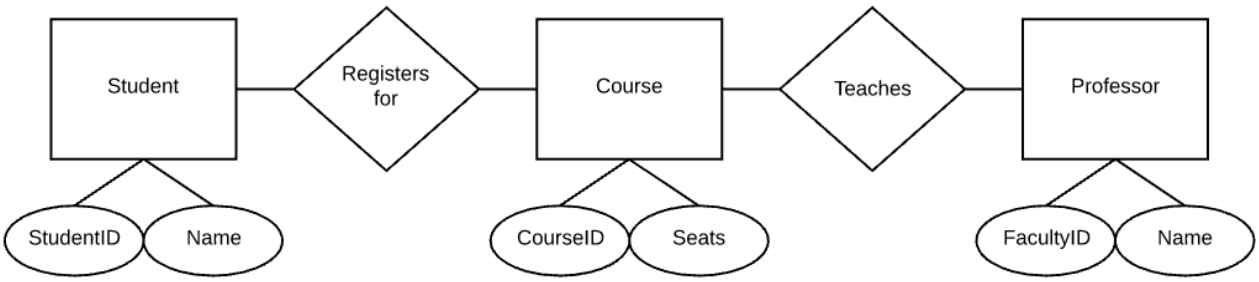
Связи обычно обозначаются глаголами, например «покупает», «содержит», «выполняет» и так далее. Взаимоотношения между тремя

указанными сущностями раскрываются посредством связок «записывается» и «преподает»:



3. Добавить атрибуты (они иллюстрируют конкретные характеристики сущности, фокусируя внимание на важной информации в контексте модели)

Атрибуты необходимы для моделирования характеристик, которые будут сопровождать каждую сущность ER-диаграммы. На практике в схемах часто встречаются такие атрибуты, как «ID-номер», «наименование» и «артикул»:



Чтобы ER-диаграмма была понятной, очень важно расположить все ее элементы в логичном порядке. Главная цель схемы «сущность-связь» состоит в том, чтобы смоделировать сложную базу данных, и поэтому важно выстраивать ER-схемы просто и логично.

Создание ER-диаграммы в Lucidchart

Lucidchart – онлайн-платформа, где удобно создавать схемы баз данных. Для работы с диаграммами необходимо пройти регистрирацию и пользоваться бесплатной версией под своей учетной записью. Lucidchart позволяет беспрепятственно упорядочивать фигуры, соединительные линии и ярлыки и

тем самым обеспечивает удобство работы по созданию ER-диаграмм. Весь процесс разворачивается онлайн, и имеется возможность коллективного редактирования схемы. Готовым результатом можно поделиться в цифровом или печатном формате.

Создав новый документ, следует активировать библиотеку фигур «сущность-связь» при помощи кнопки «+ фигуры» в левом меню (рис. 1.2):

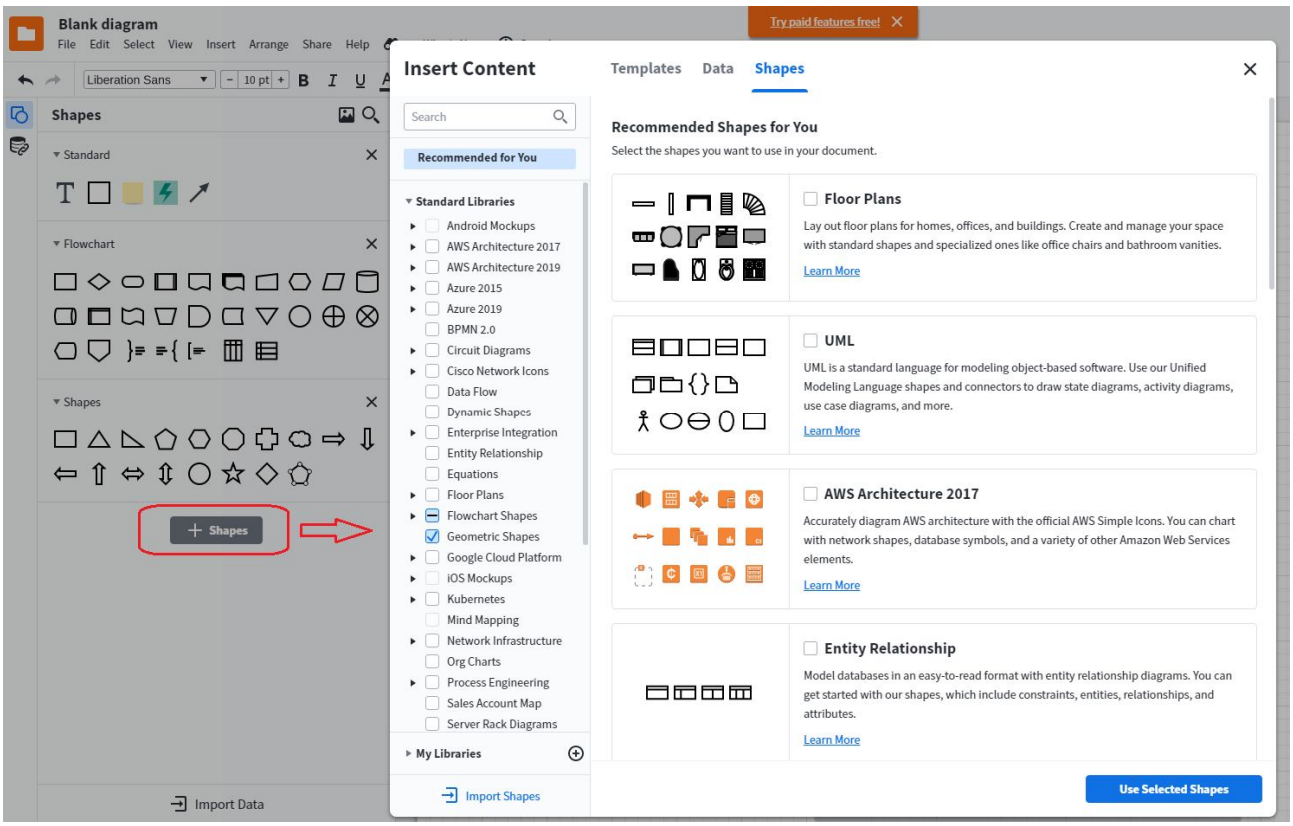


Рисунок 1.2 – Настройка панели инструментов онлайн-платформы Lucidchart

В рекомендуемых инструментах выбрать необходимые виды диаграмм, как показано на рис. 1.3.

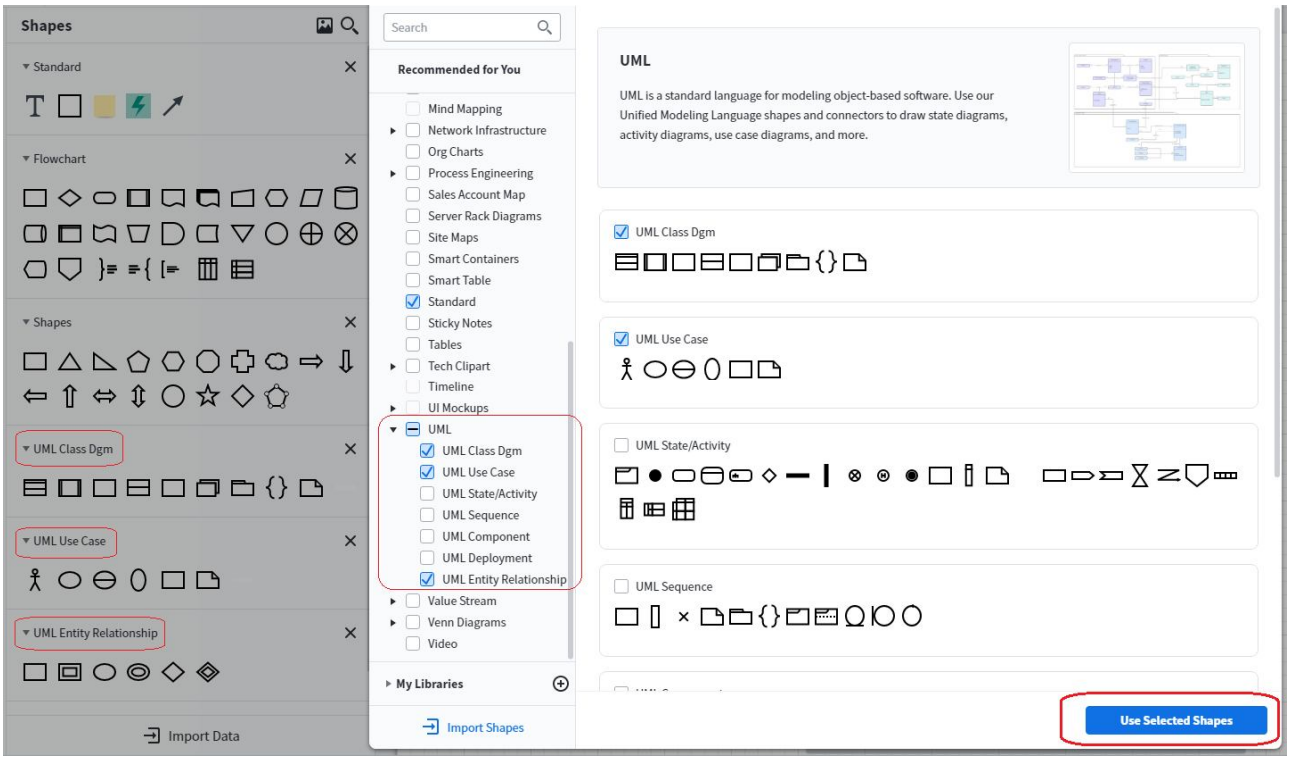


Рисунок 1.3 – Выбор необходимых диаграмм онлайн-платформы Lucidchart

Фигуры размещаются на холсте визуально (посредством перетаскивания мышью). Соединение их между собой также выполняется визуально, протянув линии из красных точек по периметру фигур. Окончания линий можно затем изменить на панели инструментов над холстом (см. рис. 1.4).

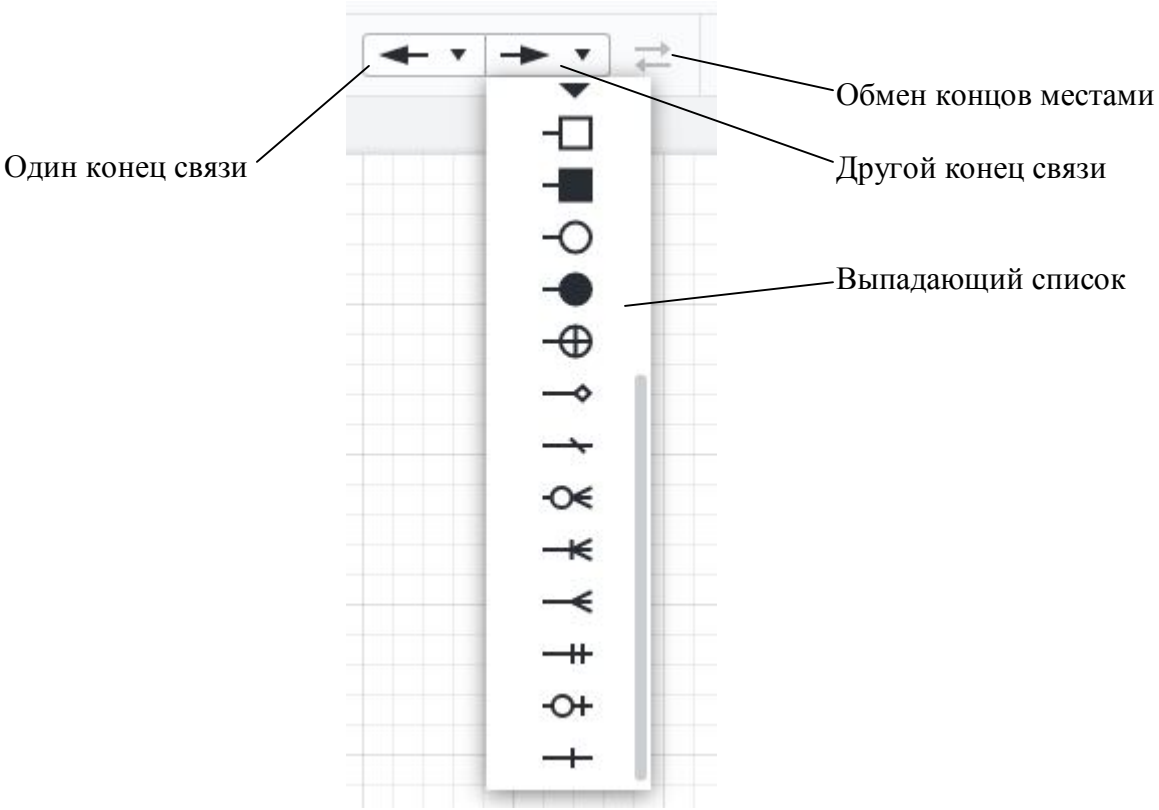


Рисунок 1.4 – Настройка связи в Lucidchart

Дополнительные советы по созданию ER-диаграмм:

1. Формировать рациональную структуру. Избегать лишних сущностей и связей.
2. Сопровождать все сущности и связи необходимыми ярлыками.
3. Обращать внимание на пробелы в связях, отсутствие сущностей или атрибутов.

Модель информационных потоков

В соответствии базовым принципом системного подхода Бизнес-процесс можно представить в виде модели «Черный ящик» (рис. 1.5).

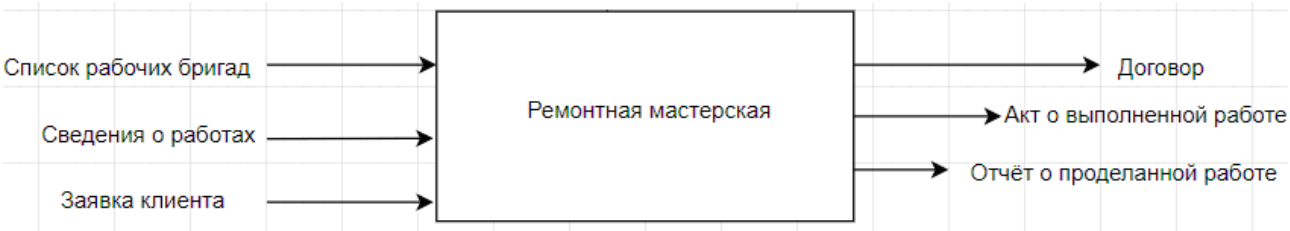


Рисунок 1.5 – Пример модели «Черный ящик» для предметной области «Ремонтная мастерская»

«Черный ящик» олицетворяет собой бизнес-процесс, протекающий в рамках предметной области. Слева в него поступают входящие информационные потоки, справа вытекают исходящие. Все эти инфопотоки образуют внешнее дополнение проектируемой программной системы, которую с некоторым приближением также можно представить этой моделью.

Все инфопотоки, а также процесс их преобразования, подлежат описанию. Для этого выполняются следующие обязательные элементы:

– приводится подробное текстовое описание бизнес-процесса;

– определяется состав действующих лиц, задействованных в рассматриваемом процессе;

– определяется входная и выходная информация, строится структурная схема типа "черный ящик" (см. рис. 1).

– производится декомпозиция бизнес-процесса на подпроцессы (задачи);

– дается общая информация о задачах;

– строится базовая модель вариантов использования (без детализации);

– выполняется описание входных и выходных информационных потоков в виде табл. 1.4 и 1.5;

Таблица 1.4 – Реестр входных информационных потоков

№	Наименование и назначение потока	Форма представле-ния*	Обработчик (Кто обрабатывает)	Корреспон-дент (Откуда)	Характеристики обработки	
					Трудозатраты, чел·ч	Периодичность, регламент

Таблица 1.5 – Реестр выходных информационных потоков

№	Наименование и назначение потока	Форма представле-ния*	Обработчик (Кто обрабатывает)	Корреспон-дент (Куда)	Характеристики обработки	
					Трудозатраты, чел·ч	Периодичность, регламент

** Возможные формы представления информации: сигнал, устное сообщение, устное распоряжение, документ, и др.*

При выделении задач следует помнить, что задача программы – это формализованная совокупность действий, выполнение которых приводит к результату заданного вида. Поэтому в качестве задач надо выбирать такие, для которых можно четко сформулировать результат. Необходимо выделить такие задачи:

1) задача ввода входных данных;
2) задача сохранения данных в памяти ЭВМ;
3) задача формирования выходных данных;
4) задача вычисления некоторого итогового показателя;
5) задача статистического анализа данных;

и т.п.

При выделении задач следует учитывать, что это информационно взаимосвязанный функциональный комплекс, назначение которого – переработка входной информацию в выходную. Вся внешняя информация представляется в форме таблиц 1.4 и 1.5.

В целом описание бизнес-процесса должно давать представление о вычислительных задачах, предполагаемых заданным бизнес-процессом, величине информационного массива, подлежащего обработке, численности персонала, занятого в процессе создания специального ПО для решения выделенных задач.

Для более строгого структурирования данного материала рекомендуется строить графические модели UML.

Модель вариантов использования UML

Модель прецедентов (модель требований, модель *вариантов использования*) – это базовая диаграмма UML, на которой изображаются отношения между действующими лицами и действиями (прецедентами). Это исходное концептуальное представление или концептуальная модель

программной системы (ПС) в процессе её проектирования и разработки. Создание диаграммы вариантов использования имеет следующие цели:

– определение общих границ и контекста моделируемой предметной области на начальных этапах проектирования ПС;

– формальное представление требований к функциональному поведению (функциональных требований) проектируемой ПС;

– разработка исходной концептуальной модели системы для ее последующей детализации в форме логических и физических моделей в нотации UML;

– подготовка исходной документации для взаимодействия разработчиков системы с ее заказчиками и пользователями.

Рассматривая данную диаграмму в качестве модели бизнес-системы, можно ассоциировать ее с "черным ящиком" (см. рис. 1.5). Концептуальный характер этой диаграммы проявляется в том, что подробная детализация диаграммы на начальном этапе проектирования имеет отрицательный характер, поскольку предопределяет способы реализации поведения системы – эти аспекты должны быть сознательно скрыты от разработчика на модели прецедентов.

Базовыми элементами данной диаграммы являются вариант использования и актер.

Вариант использования (use case) – внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с актерами, представляет собой спецификацию общих особенностей поведения или функционирования моделируемой системы без рассмотрения ее внутренней структуры.

Отдельный вариант использования обозначается на диаграмме эллипсом, внутри которого содержится его краткое имя в форме глагольного существительного (рис. 1.6, а) или глагола (рис. 1.6, б) с пояснительными словами. Сам текст имени прецедента должен начинаться с заглавной буквы.

Имя (name) – строка текста, которая используется для идентификации любого элемента модели.

а) глагольное существительное б) глагол в) актер

Рисунок 1.6 – Графическое обозначение варианта использования и актера на модели вариантов использования UML

Примерами вариантов использования могут быть следующие действия:

– занесение систему данных о клиенте;
– проверка состояния текущего счета клиента;
– оформление заказа на покупку товара;
– получение информации о кредитоспособности клиента;
– отображение графической формы на экране монитора;
– авторизация пользователя;

и другие действия, процессы, задачи.

Актер (actor) – согласованное множество ролей, которые играют внешние сущности по отношению к вариантам использования при взаимодействии с ними, представляет собой любую внешнюю по отношению к моделируемой системе сущность, которая взаимодействует с системой и использует ее функциональные возможности для достижения определенных целей или решения частных задач.

Каждый актер может рассматриваться как определенная роль относительно конкретного варианта использования. Стандартным графическим обозначением актера на диаграммах является фигурка человечка, под которой записывается имя актера (рис. 1.6, в). Имя актера должно быть достаточно информативным с точки зрения семантики. Это может быть наименование должности (например, продавец, кассир, менеджер, и т.д.).

Пример модели вариантов использования показан на рисунке 1.7.

Рисунок 1.7 – Пример модели вариантов использования для предметной области «Ремонтная мастерская»

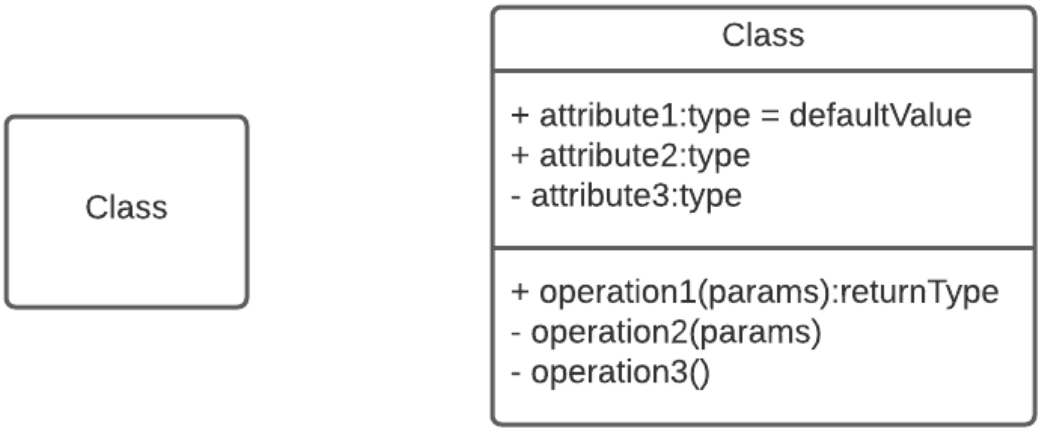
Модель классов UML

Диаграмма классов отражает различные взаимосвязи между отдельными сущностями предметной области, а также описывает их внутреннюю структуру и типы отношений. Вместе с тем на данной диаграмме не указывается информация о временны́х аспектах функционирования ПС. С этой точки зрения диаграмма классов может выступать дальнейшим развитием концептуальной модели проектируемой системы.

Диаграмма классов (class diagram) – диаграмма языка UML, на которой представлена совокупность декларативных или статических (не зависящих от времени) элементов модели, таких как классы с атрибутами и операциями, а

также связывающие их отношения, и предназначена для представления статической структуры модели системы в терминологии классов ООП.

Класс (class) – абстрактное описание множества однородных объектов, имеющих одинаковые атрибуты, операции и отношения с объектами других классов. Графически класс в нотации UML изображается в виде прямоугольника, который дополнительно может быть разделен горизонтальными линиями на разделы или секции (рис. 1.8). На начальных этапах разработки диаграммы отдельные классы могут обозначаться простым прямоугольником, в котором должно быть указано его имя (рис. 1.8, а). По мере проработки отдельных компонентов диаграммы описания классов дополняются атрибутами и операциями (рис. 1.8, б).



(а) (б)

Рисунок 1.8 – Графическое отображение класса в Lucidchart: а – на начальном этапе разработки, б – после тщательной проработки.

Имя класса должно быть уникальным в пределах пакета, который может содержать одну или несколько диаграмм классов. Имя указывается в самой верхней секции прямоугольника, поэтому она называется секцией имени класса. Имя класса записывается по центру секции полужирным шрифтом и должно начинаться с заглавной буквы. Рекомендуется в качестве имен классов использовать имена существительные без пробелов. Необходимо помнить, что имена классов образуют словарь предметной области при проектировании ПС.

В секции имени класса могут также находиться стереотипы или ссылки на стандартные шаблоны, от которых образован данный класс и, соответственно, от которых он наследует свои свойства: атрибуты и операции.

Атрибут (attribute) – содержательная характеристика класса, описывающая множество значений, которые могут принимать отдельные объекты этого класса. Атрибут класса служит для представления отдельного свойства или признака, который является общим для всех объектов данного класса. Атрибуты класса записываются в секции атрибутов (рис. 1.8, б).

Операция (operation) – это сервис, предоставляемый каждым экземпляром или объектом класса по требованию своих клиентов, в качестве которых могут выступать другие объекты, в том числе и экземпляры данного класса. Операции класса записываются в секции операций (рис. 1.8, б). Совокупность операций характеризует функциональный аспект поведения всех объектов данного класса, который описывается диаграммами поведения UML. Имя операции представляет собой строку текста, которая используется в качестве идентификатора соответствующей операции и должна быть уникальной в пределах данного класса. Имя операции должно начинаться со строчной буквы, и, как правило, записываться без пробелов.

Идентификация классов анализа

Идентификация классов анализа заключается в предварительном определении набора классов на основе описания предметной области.

Способы идентификации основных абстракций аналогичны способам идентификации сущностей в модели "сущность-связь" – поиск абстракций, описывающих физические или материальные объекты, процессы и события, роли людей, организации и другие понятия предметной области. Единственным формальным способом идентификации сущностей является анализ текстовых описаний предметной области, выделение из описаний имен существительных и выбор их в качестве "кандидатов" на роль абстракций.

Каждая сущность должна иметь наименование, выраженное существительным в единственном числе.

Далее абстракции идентифицируются по трем типам, которые используются для уточнения семантики отдельных классов при построении различных диаграмм:

1. Управляющий класс (стереотип <<control>>) – класс, отвечающий за координацию действий других классов на диаграмме. На каждой диаграмме классов должен быть хотя бы один управляющий класс, который контролирует последовательность выполнения действий данного варианта использования или ПС целиком. Данный класс является активным и инициирует рассылку множества сообщений другим классам модели. Примеры управляющих классов: *менеджер транзакций, журнал операций, координатор ресурсов, обработчик событий, обработчик ошибок*, и т.п.

2. Класс-сущность (стереотип <<entity>>) – пассивный класс, информация о котором должна храниться постоянно и не уничтожаться с выключением системы или завершением работы программы. Как правило, *этот класс соответствует отдельной сущности логической модели данных*. Этот класс лишь принимает сообщения от других классов модели, реагируя на них посредством своих операций. Источники выявления классов-сущностей: сущности ERD, глоссарий проекта, описание потоков событий вариантов использования.

3. Граничный класс (стереотип <<boundary>>) – класс, который располагается на границе ПС с внешней средой и непосредственно взаимодействует с актерами, являясь при этом составной частью ПС. Как правило, для каждой пары "действующее лицо – вариант использования" определяется один граничный класс. Типы граничных классов:

– пользовательский интерфейс, выражающий обмен информацией с пользователем без деталей интерфейса: кнопок, списков, окон, и т.п.

– системный интерфейс с надсистемой или другой системой (используемые протоколы обмена без деталей их реализации);

– аппаратный интерфейс для связи с внешними техническими устройствами (аппаратные средства, протоколы, драйверы).

Отношения между классами

Классы вступают друг с другом в отношения. На диаграмме классов выделяют такие виды отношений: ассоциация, наследование, агрегирование и использование. При изображении конкретной связи ей можно сопоставить текстовую метку, документирующую имя этой связи и/или ее роль. Имя отношения должно быть уникально в своем контексте. Также связь может иметь по ее концам обозначения кратности (множественности) отношений между классами, некоторые из которых показаны ни рис. 1.9. Кратность отношений может быть задана для всех типов отношений, кроме наследования.

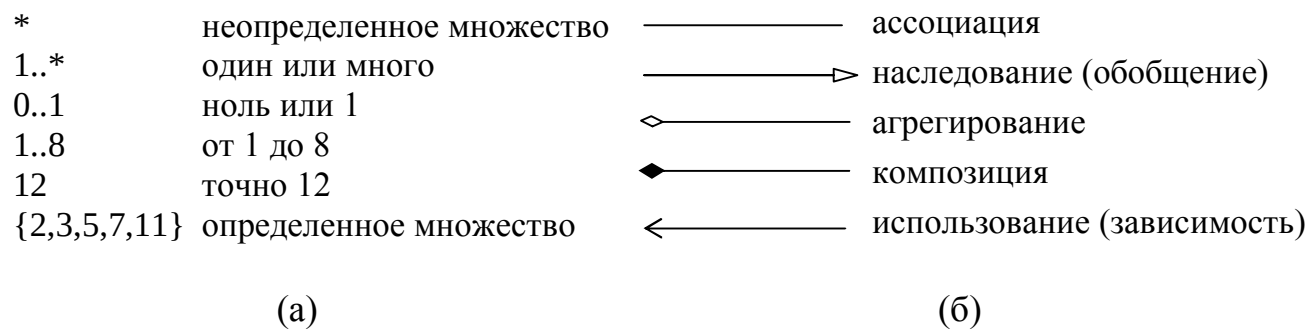


Рисунок 1.9. Обозначения отношений между классами на диаграмме классов UML: а – множественности, б – значки типов отношений

Ассоциация соединяет два класса и означает наличие семантической связи между ними. Ассоциации часто отмечаются существительными (например, Выбор), описывающими природу связи. Одна пара классов может иметь одну или несколько ассоциативных связей.

Наследование представляет отношение типа "общее/частное", выглядит как значок ассоциации с большой незакрашенной треугольной стрелкой на стороне суперкласса. Наследование может быть одиночное или множественное.

Агрегирование/Композиция обозначает отношение "целое/часть" и направлена круглой или ромбической стрелкой к агрегату. Экземпляры класса на другом конце стрелки будут в определенном смысле частями экземпляров класса-агрегата. Разрешается рефлексивная и циклическая агрегация.

Агрегация не требует обязательного физического включения части в целое, композиция – требует. Части композита живут и уничтожаются вместе с ним.

Использование обозначает отношение типа "клиент/сервер" и изображается как ассоциация с открытой стрелкой, соответствующем клиенту. Эта связь выражает зависимость клиента от сервера и означает, что клиент нуждается в услугах сервера, то есть операции класса-клиента вызывают операции класса-сервера или имеют сигнатуру, в которой возвращаемое значение и/или аргументы принадлежат классу сервера. Фактически отношение использования показывает, что один элемент использует другой.

Общие рекомендации по построению диаграмм классов:

– использовать зависимость, только в случае, если моделируемое отношение не является структурным;

– использовать обобщение, только если имеет место отношение типа "является";

– располагать элементы так, чтобы свести к минимуму число пересекающиеся линий отношений;

– пространственно организовывать элементы так, чтобы семантически близкие сущности располагались рядом;

– чтобы привлечь внимание к важным особенностям диаграммы, использовать примечания (комментарии) и выделение цветом;

Пример модели классов показан на рисунке 1.10. На этой диаграмме представлены все типы классов и отношений между ними. Граничные и управляющие классы по отношению к классам-сущностям выступают как серверы.

Контрольные вопросы и упражнения

1. Что такое ER-модель?
2. Для чего предназначена диаграмма классов UML?
3. В чем принципиальные сходства между ER-моделью и диаграммой классов?

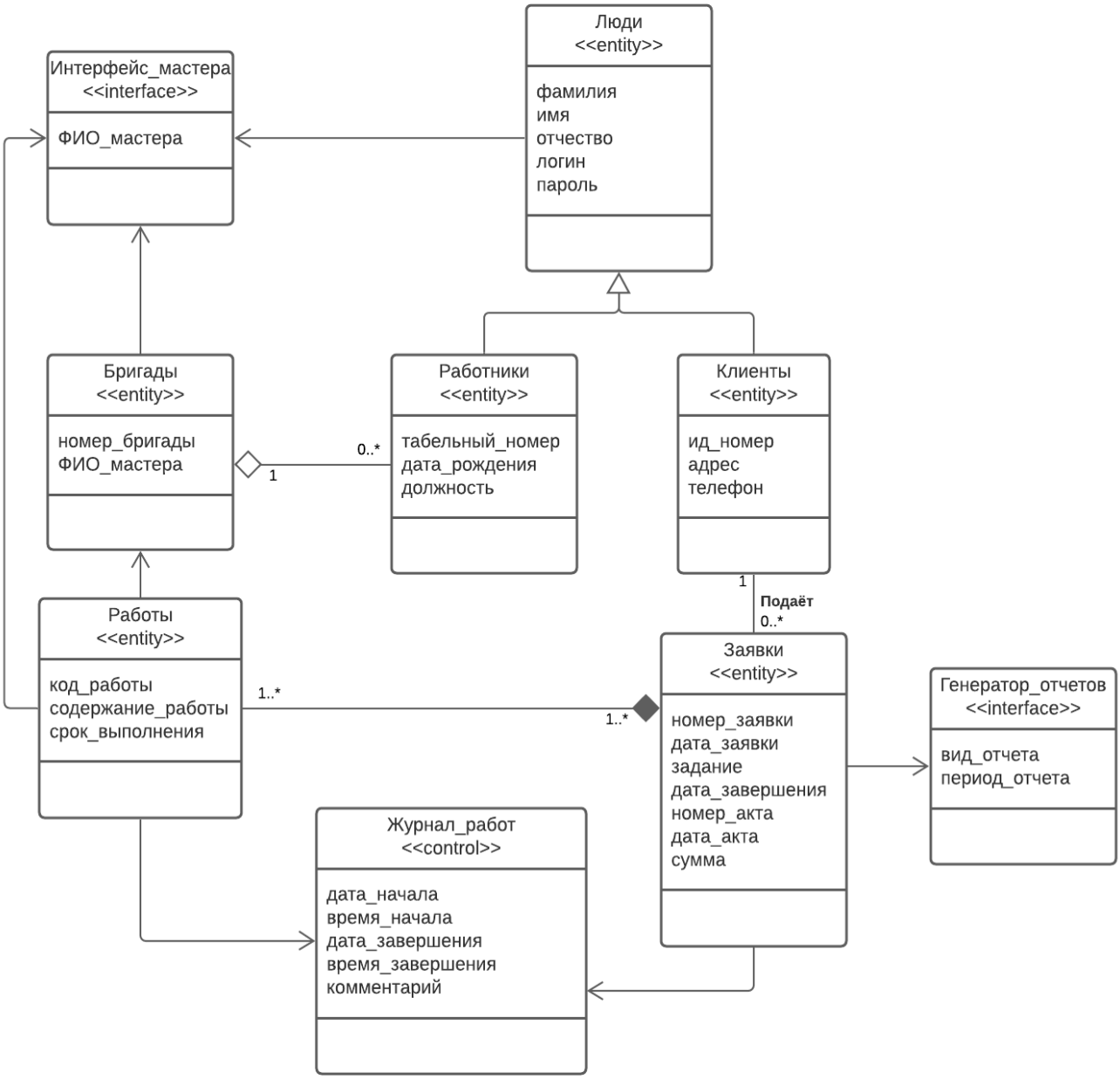


Рисунок 1.10. Диаграмма классов для предметной области «Ремонтная мастерская», построенная в Lucidchart

4. В чем принципиальные отличия между ER-моделью и диаграммой классов?
5. Как связаны между собой модели UML вариантов использования и классов?
6. Для заданной предметной области постройте модель «Черный ящик» и сформируйте реестры входных и выходных данных.
7. Для заданной предметной области постройте диаграмму классов с использованием всех возможных типов отношений между классами.